LECTURE 3
DATA ENCRYPTION STANDARD (DES)

DR. NERMIN HAMZA

FROM CLASSICAL CIPHERS TO MODERN CIPHERS

- Classical Cipher→ Modern Cipher (Block cipher)
  - Alphabetic letters → Binary data
    - Easy to do
  - Large block size, large key space
- DES=
  - Substitution + Transposition + Feistel Network

BLOCK CIPHER PRINCIPLES

- most symmetric block ciphers are based on a **Feistel Cipher Structure**
- needed since must be able to **decrypt** ciphertext to recover messages efficiently
- would need table of 2^{64} entries for a 64-bit block
- instead create from smaller building blocks
- using idea of a product cipher

CLAUDE SHANNON AND SUBSTITUTION-PERMUTATION CIPHERS

- Claude Shannon introduced idea of substitution-permutation (S-P) networks in 1949 paper
- form basis of modern block ciphers
- S-P nets are based on the two primitive cryptographic operations seen before:
  - *substitution* (S-box): Each plaintext group of elements is uniquely replaced by a corresponding ciphertext group of elements
  - *permutation* (P-box): A sequence of plaintext elements is replaced by a permutation of that sequence . No added , deleted or replaced in the sequence.
- provide *confusion* & *diffusion* of message & key

CONFUSION AND DIFFUSION

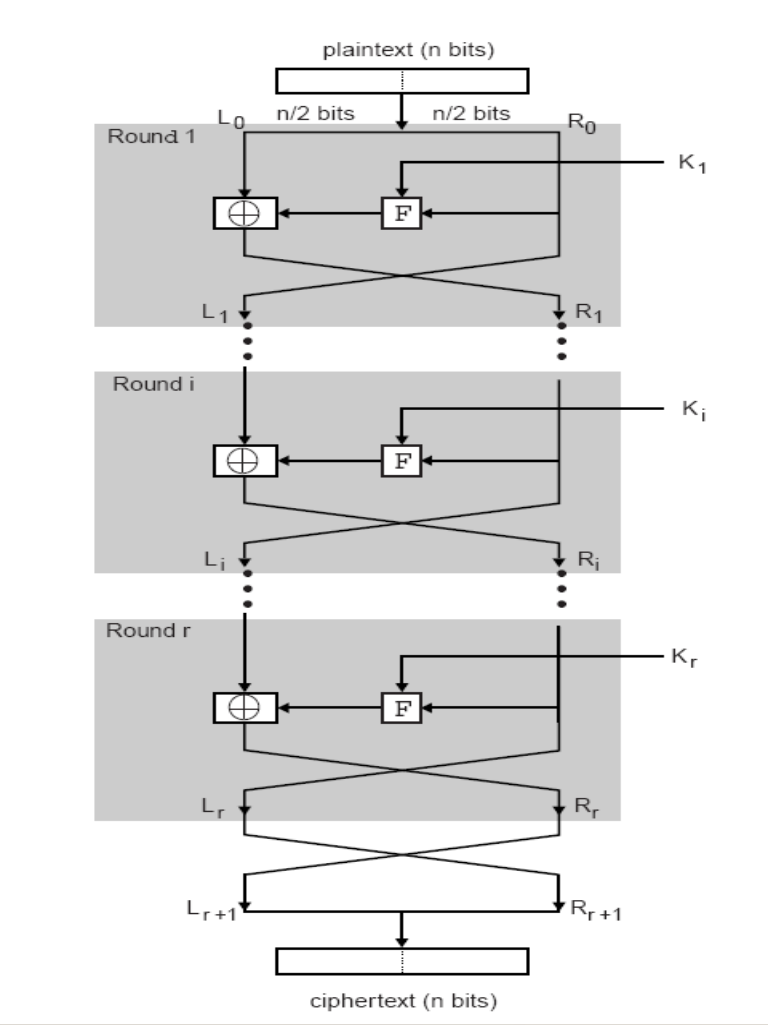
- cipher needs to completely obscure statistical properties of original message
- a one-time pad does this
- more practically Shannon suggested combining S & P elements to obtain:
- **diffusion** – dissipates تبدد statistical structure of **plaintext** over bulk of **ciphertext**
- **confusion** – makes relationship between **ciphertext** and **key** as complex as possible

CONFUSION AND DIFFUSION

- Diffusion can be accomplished by having many plaintext characters affect each ciphertext character.
- An example of diffusion is the encryption of a message $M=m_1,m_2,\ldots$ using a an averaging: $y_n = \square_{=1,k} m_{n+i} (\mathrm{mod}\, 26)$.

- Diffusion complicates the statistics of the ciphertext, and makes it difficult to discover the key of the encryption process.
- The process of **confusion**, makes the use of the key so complex, that even when an attacker knows the statistics, it is still difficult to deduce the key.

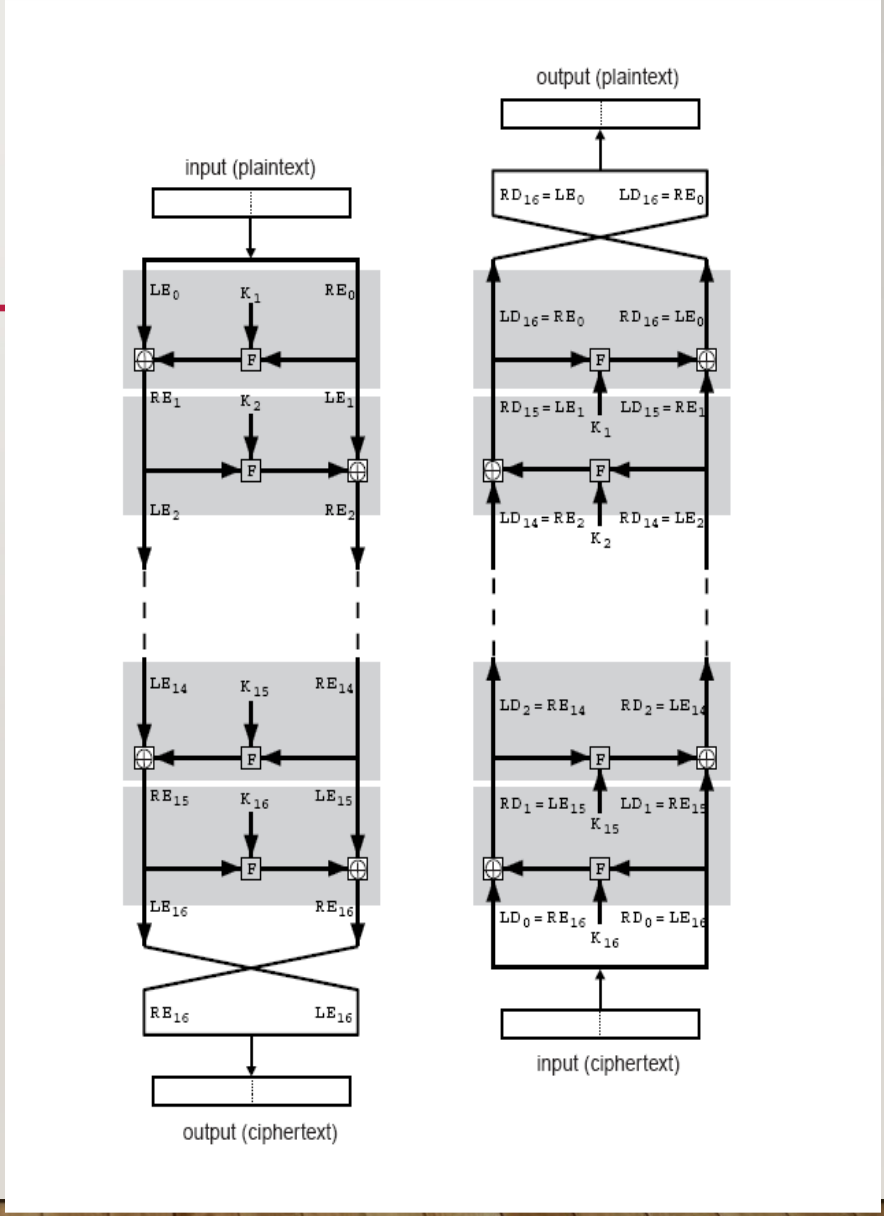
Feistel Network



- Design features/parameters
  - Block size
  - Key size
  - Number of rounds
  - Subkey generation algorithm
  - Round function (F)

FE

- The process of decryption with a Feistel cipher is essentially the same as the encryption process.
- Rule:
  - Use the ciphertext as input to the algorithm, but use the subkeys in the reverse order



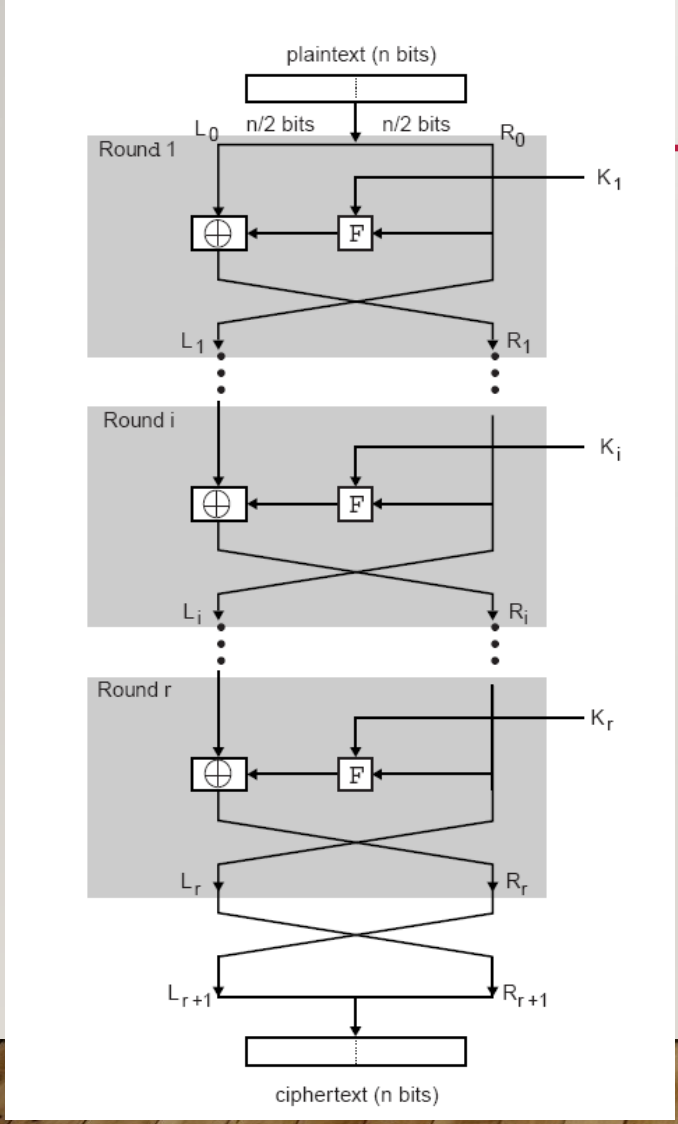
MAIN MODERN CRYPTOGRAPHY ALGORITHM

- **DES**: (Data Encryption Standard), was the first encryption standard to be recommended by NIST (National Institute of Standards and Technology). It is based on the IBM proposed algorithm called Lucifer.
- **3DES:** As an enhancement of DES, the3DES (Triple DES) encryption standard was proposed. In this standard the encryption method is similar to the one in original DES but applied 3 times to increase the encryption level.
- **AES:** (Advanced Encryption Standard), is the new encryption standard recommended by NIST to replace DES..
- **Blowfish:** Blowfish is a variable length key, 64-bit block cipher. The Blowfish algorithm was first introduced in 1993. This algorithm can be optimized in hardware applications though it's mostly used in software applications.

DES OVERVIEW

- Data Encryption Standard (DES)
  - Most widely used encryption scheme
  - Adopted as Federal Information Processing Standard (FIPS) for the United States in 1976
  - 64-bit data block
  - 56-bit key
  - Based on Feistel Network
    - The decryption follows the same process as the encryption.
  - Now considered insecure for many application

DES STRUCTURE



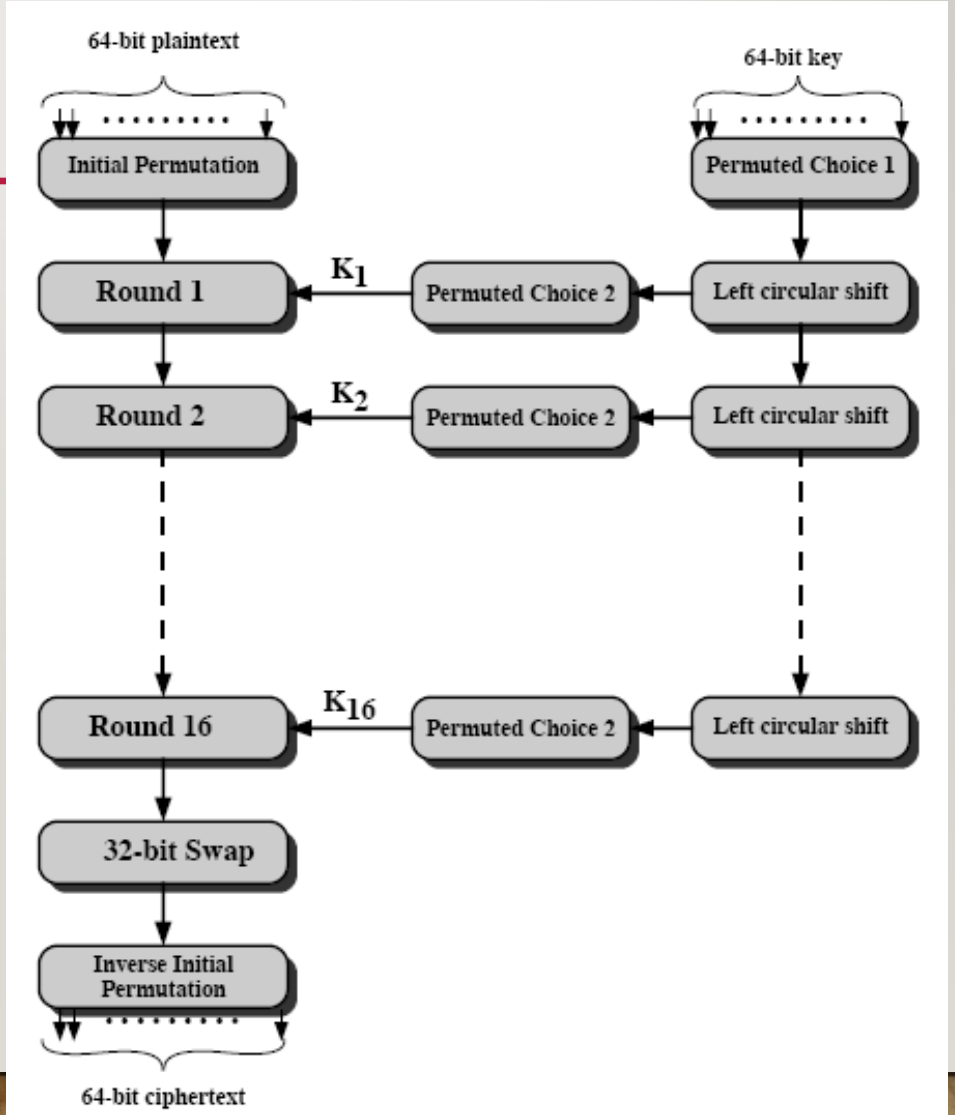


Figure 3.7 General Depiction of DES Encryption Algorithm

DES…

- **Initial Permutation (IP):**

  The plaintext block undergoes an intial permutation.

  64 bits of the block are permuted.

- **A Complex Transformation:**

  64 bit permuted block undergoes 16 rounds of complex transformation. (Using subkeys)

DES…

- **32-bit swap:**

32 bit left and right halves of the output of the 16th round are swapped.

- **Inverse Initial Permutation (IP-1):**

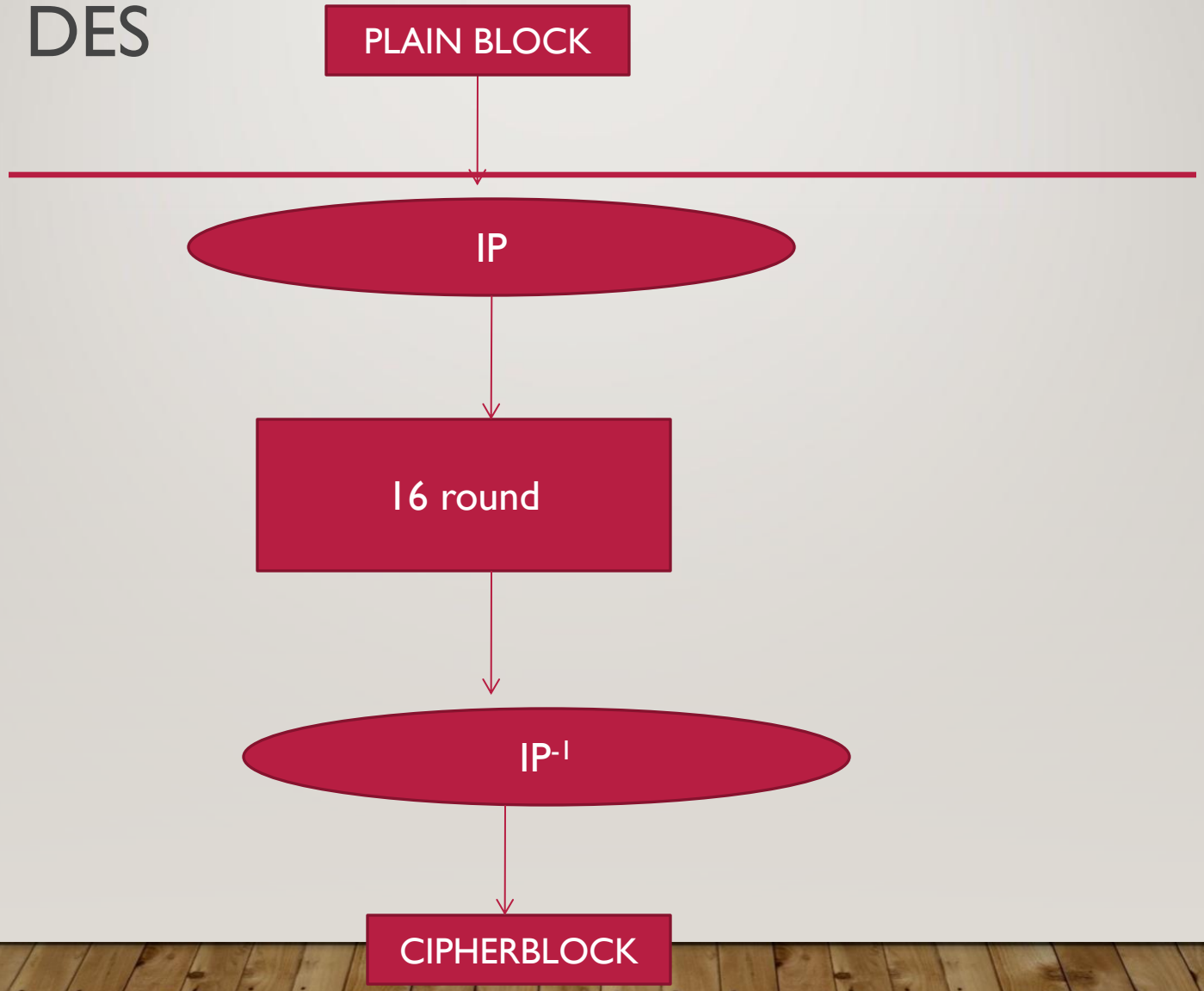
The 64 bit output undergoes a permutation that is inverse of the intial permutation.

The 64 bit output is the ciphertext.

DES: IP

- WHAT IS IP ?
- IP FOR initial permutation.
- 64 bits ➔ 64 bits
- According to IP table

DES



IP TABLE

IP

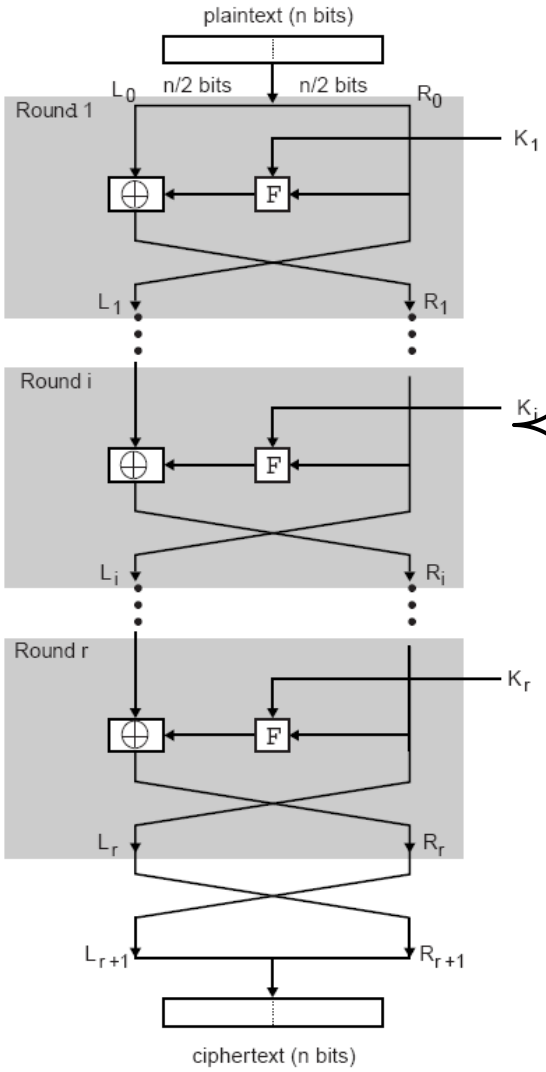
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

INVERSE PERMUTATION

IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

SINGLE ROUND



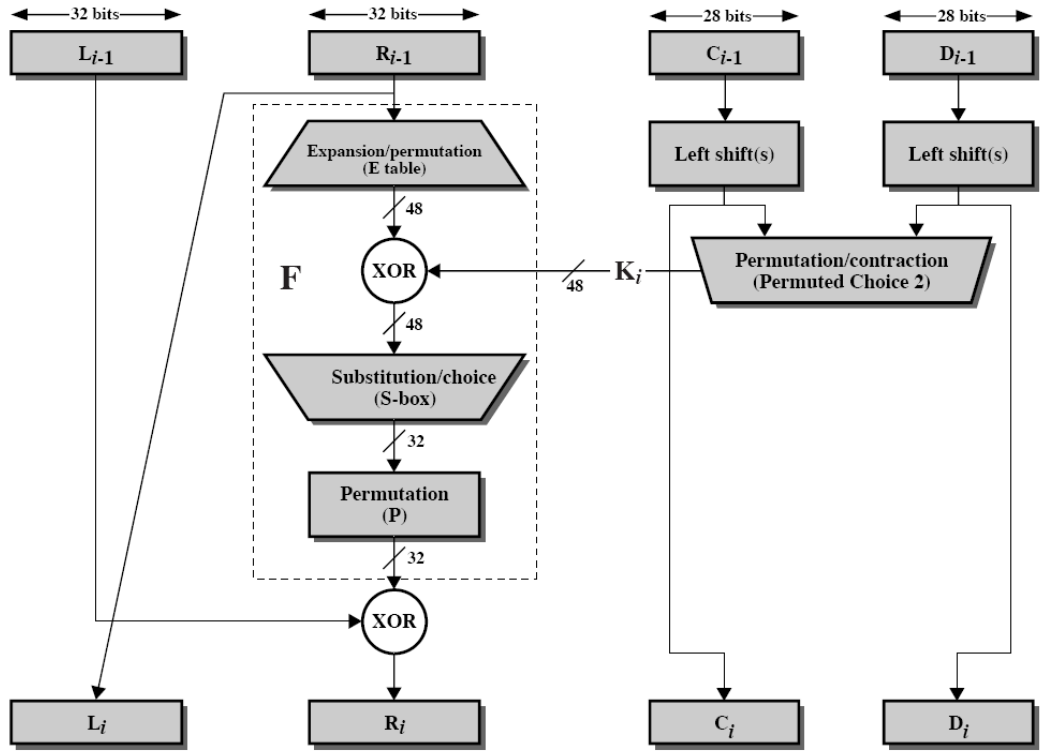


Figure 3.8 Single Round of DES Algorithm

LET US TAKE AN EXAMPLE

- Let M=

0000 0001 0010 0011 0100 0101 0110 0111

1000 1001 1010 1011 1100 1101 1110 1111

So

What happen after Initial Permutation

IP? 1100 1100 0000 0000 1100 1100
1111 1111 1111 0000 1010 1010
1111 0000 1010 1010

58 50 42 34 1100

- IP :

1100 1100 0000 0000 1100 1100 1111 1111

1111 0000 1010 1010 1111 0000 1010 1010

L_0 = 1100 1100 0000 0000 1100 1100 1111 1111

R_0 = 1111 0000 1010 1010 1111 0000 1010 1010

SINGLE ROUND

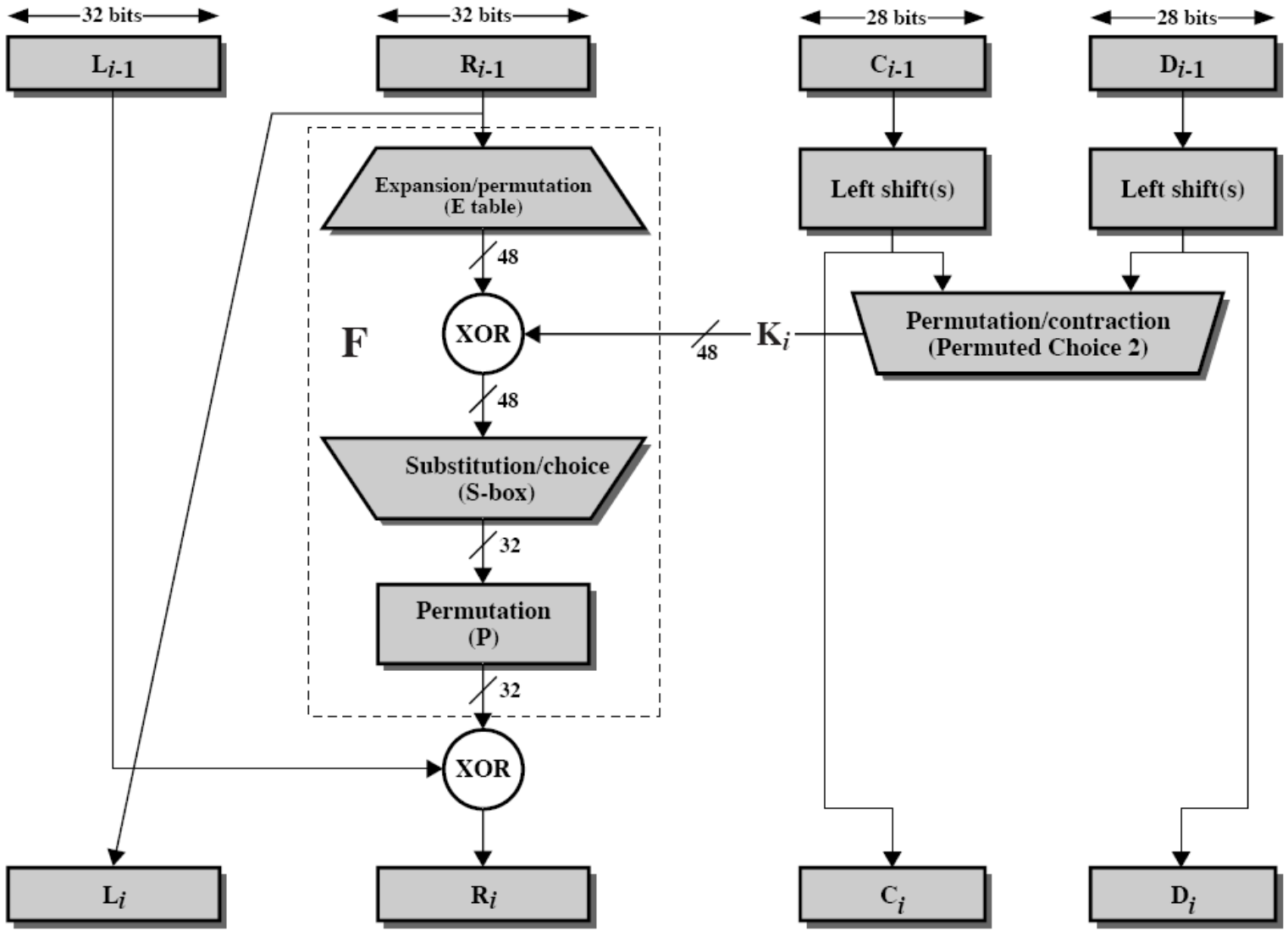


Figure 3.8 Single Round of DES Algorithm

SINGLE ROUND

- From the picture:

  $L_n = R_{n-1}$
  $R_n = L_{n-1}$ xor $f(R_{n-1}, K_n)$

- For $n = 1$, we have

  K_1 = 000110 110000 001011 101111

  111111 000111 000001 110010

  $L_1 = R_0$ = 1111 0000 1010 1010

  1111 0000 1010 1010

  $R_1 = L_0$ xor $f(R_0, K_1)$

SINGLE ROUND :WHAT IS F?

- $R_n = L_{n-1}$ xor $f(R_{n-1}, K_1)$
- **Note: R is 32 and K is 48 what happen?**

- ***What*** $f(R_{n-1}, K_n)$***?***

Details of function F:

It takes 32 bits input and produces a 32 bit output

SINGLE ROUND :WHAT IS F?

- **E(*Rn-1*)→ E BIT-SELECTION TABLE** …. For expanding 32 to be 48
- K_1 ***xor*** **E**(R_{n-1}) ….Actually 48 bits
- K_n xor **E**(R_{n-1}) $=B_1B_2B_3B_4B_5B_6B_7B_8$,

  … divide to 8 block each one =6

- $S_i(B_i) = S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$

  perform S box substitution

- f= **P**($S_1(B_1)S_2(B_2)...S_8(B_8)$)

  … to come back 32 with using P permutation table

SINGLE ROUND :WHAT IS F?

- **Details of function F:**

32 bit input is expanded into 48 bits.

-This is done by permuting and

duplicating some bits of 32 bits.

Exclusive OR operation is performed between these 48 bits and 48 bit subkey.

SINGLE ROUND :WHAT IS F?

- **Details of function F (cont...):**

48 bit output of the Exclusive OR operation is grouped into 8 groups of 6 bits each.

Each 6 bit group is fed into a 6-to-4 substitution box that transforms 6 bits to 4 bits.

SINGLE ROUND :WHAT IS F?

- **Details of function F:...**

32 bit output of 8 substitution boxes is fed into a permutation box.

The 32 bit output of the permutation box is $F(R_{i-1}, K_i)$.

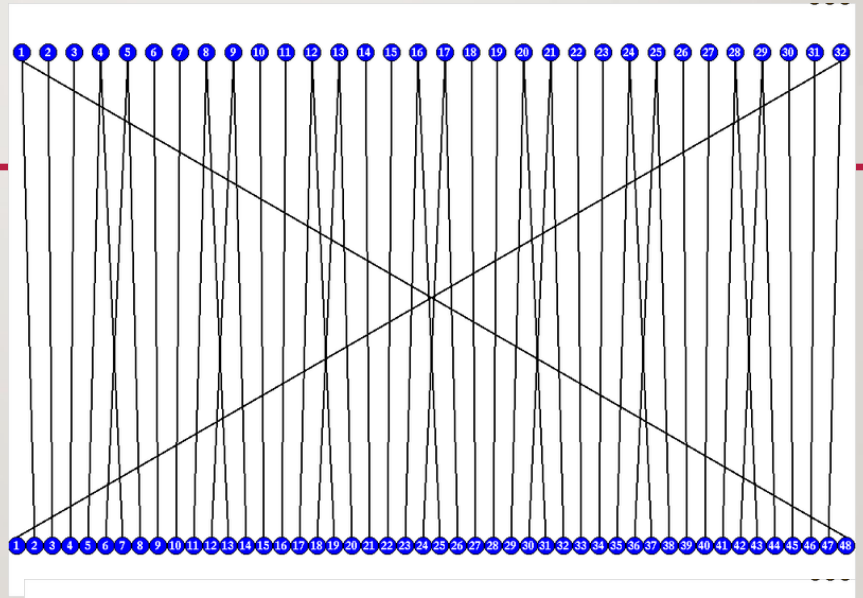
EXPANSION (E) AND PERMUTATION (P)

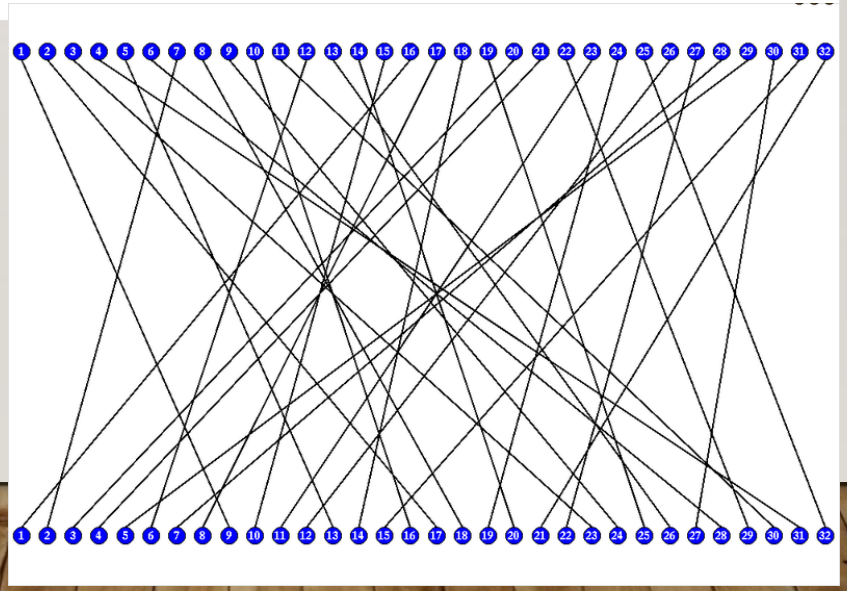
E

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25





S-BOX

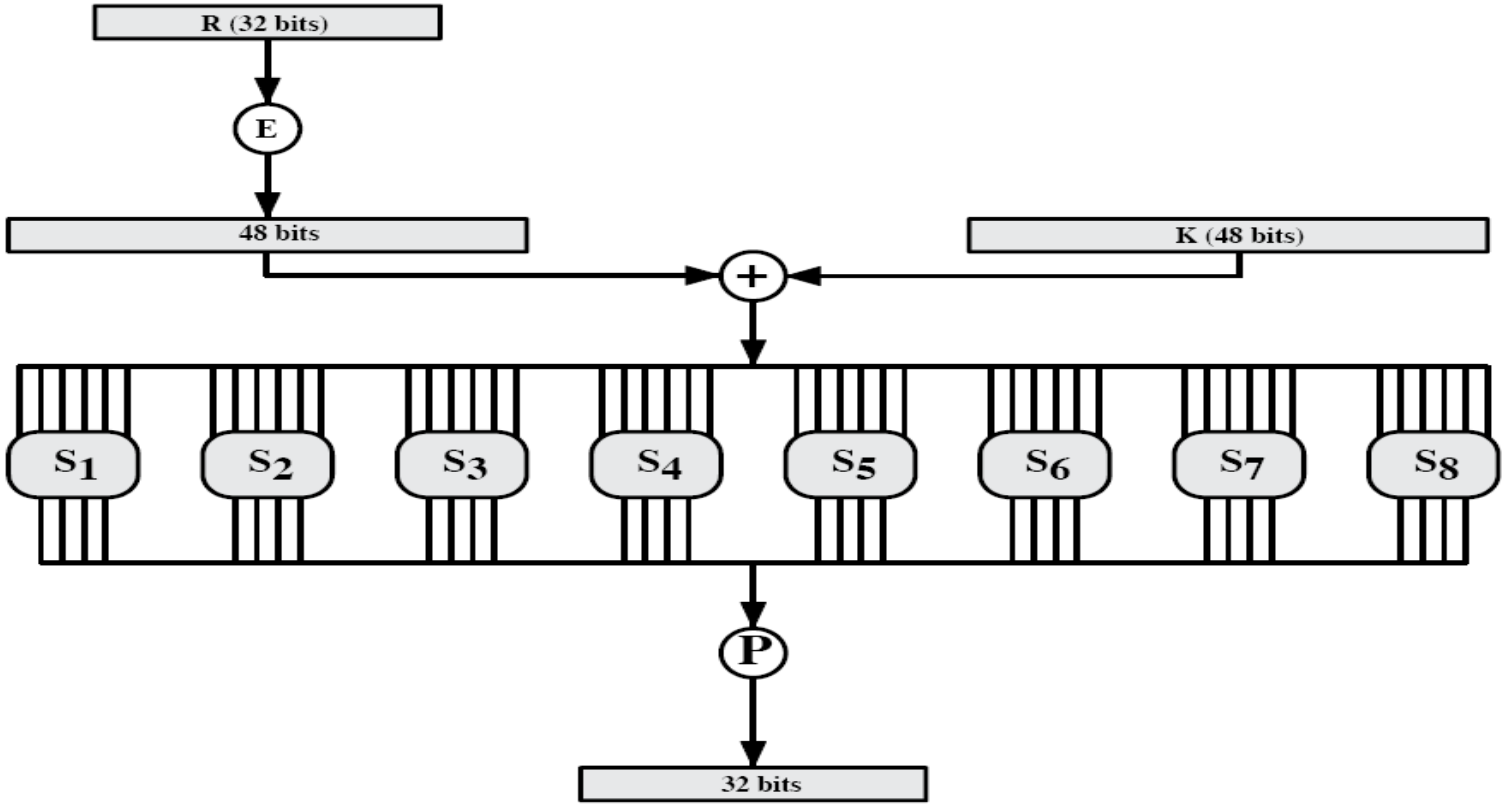


Figure 3.9 Calculation of F(R, K)

S-BOX DEFINITION

S_1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S-BOX DEFINITION

S_5	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

BIT SELECTION EXPANSION

- R_0 = 1111 0000 1010 1010 1111 0000 1010 1010
  from 32 to 48 bits
- $E(R_0)$ =

011110 100001

010101 010101

011110 100001

010101 010101

E BIT-SELECTION TABLE

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

XOR KEY WITH E(R_{N-1})

K_n xor $\mathbf{E}(R_{n-1})$.

K_1 =	000110 110000 001011 101111
	111111 000111 000001 110010
$\mathbf{E}(R_0)$ =	011110 100001 010101 010101
	011110 100001 010101 010101
K_1 xor $\mathbf{E}(R_0)$ =	011000 010001 011110 111010
	100001 100110 010100 100111

DES: USING S-BOX

- K_n xor $\mathbf{E}(R_{n-1}) = B_1B_2B_3B_4B_5B_6B_7B_8$,

- $S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$

K_1 ***xor*** $\mathbf{E}(R_0)$ =

011000 010001 011110 111010

100001 100110 010100 100111

- B1= 011000
- S1(B1)= ?

DES: USING S-BOX

S1

Column Number

Row No.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

$S_1(011000) = 0101.$

HOW?

DES: USING S-BOX

- 011000
- 00 is the row : 0
- 1100 the Colum : 12
- From the table 0 and 12 will be 5 which is 0101

S1

Column Number

Row No.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

DES: USING S-BOX

- K_1 xor $\mathbf{E}(R_0)$ =

011000 010001 011110 111010

100001 100110 010100 100111.

- $S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$ =

0101 1100 1000 0010

1011 0101 1001 0111

DES: USING P TABLE

$$f = P(S_1(B_1)S_2(B_2)\ldots S_8(B_8))$$

0101 1100 1000 0010
1011 0101 1001 0111

16 7 20 21
0 0 1 0

f =
0010 0011 0100 1010
1010 1001 1011 1011

P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

DES: USING INVERSE PERMUTATION IP TABLE

- After 16 round ?

- Block = $R_{16}L_{16}$

Perform inverse permutation **IP**$^{-1}$

IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

DES: USING INVERSE PERMUTATION IP TABLE

- then ***reverse*** the order of the two blocks into the 64-bit block : $R_{16}L_{16}$
- $R_{16}L_{16}$ =

00001010 01001100 11011001 10010101

01000011 01000010 00110010 00110100

IP^{-1} =

10000101 11101000 00010011 01010100

00001111 00001010 10110100 00000101

DES ANIMATION

- https://www.youtube.com/watch?v=Vcld7CMAnNs&t=204s